

① Star Patterns :-

(a) Square :-

Logic :

- Use two nested loops.
- The outer loop controls the rows (running n times).
- The inner loop controls the columns (running n times).
- Print $*$ in every iteration of inner loop and move to a new line after the inner loop finishes.

Code :

```
int n = 5;  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.println(" *");  
    }  
    System.out.println();  
}
```

(b) Hollow Square :-

Logic :

- Use two nested loops running n times each.
- Check the current row and column indices.
- If it is the first row, last row, first

- column, or last column, print *.
- Otherwise, print a blank space " " to create the hollow center.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i == 0 || i == n-1 || j == 0 || j == n-1) {
            System.out.println("*");
        }
        else {
            System.out.println(" ");
        }
    }
    System.out.println();
}
```

(c) Triangle :-

Logic :-

- Use an outer loop for n rows.
- Use an inner loop that runs from 0 to up to the current row index i.
- Print * in the inner loop to gradually increase the number of stars per row.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

(d) Inverted Triangle :-Logic :

- Use an outer loop to track the rows, running n times (from 0 to $n-1$).
- Use a first inner loop to print the leading spaces. The number of spaces starts at 0 and increases by 1 on each new row (i spaces).
- Use a second inner loop to print the stars. To keep the triangle centered, the stars must decrease by 2 each time.
- Move to the next line after both inner loops finish.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
}

```

```

    }
    for (int j = 0; j < n - i; j++) {
        System.out.println(" ");
    }

```

```

    System.out.println();
}

```

(e) Pyramid :-

Logic :-

- Use an outer loop for n rows.
- Use a first inner loop to print spaces.
- The number of spaces decrease as the row number increases ($n - i - 1$).
- Use a second inner loop to print stars.
- The number of stars follow odd numbers.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        System.out.println(" ");
    }
    for (int j = 0; j < 2 * i + 1; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

(f) Diamond :-Logic :

- Combine a standard pyramid logic for the upper half and an inverted pyramid logic for the lower half.
- The upper half loop runs n times.
- The lower half loop runs $n-1$ times, with spaces increasing and stars decreasing.

Code :

```

int n = 5;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * i + 1; j++) {
        System.out.print("*");
    }
    System.out.print("\n");
}

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * (n - i - 1) - 1; j++) {
        System.out.print("*");
    }
}

```

```

    System.out.println();
}

```

(g) Half Diamond :-

Logic :

- Create an increasing right triangle for the upper half.
- Create decreasing inverted right triangle for the lower half.
- The upper loop increases stars from 1 to n. The lower loop decreases stars from n-1 to 1.

Code :

```

int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
for (int i = n-1; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

(h) Half Diamond Inverted :-Logic :

- Use two main segments : Upper and lower.
- In each segment, use an inner loop to print the required leading spaces, followed by a loop to print the stars.

Code :-

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < n - i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

② Number Patterns :-

(a) Square :-

Logic :

- Use two nested loops, both running n -times.
- Instead of a star, strictly print the number 1 during every iteration of inner loop.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print("1");
    }
    System.out.println();
}
```

(b) Incrementing :-

Logic :

- Use an outer loop for n rows.
- Use an inner loop that runs from 1 to n .
- Print the value of inner loop variable to create an incrementing sequence on each row.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print(j);
    }
    System.out.println();
}
```

(c) Right Triangle :-

Logic :

Use an outer loop running from 1 to n to represent the row number.
Use an inner loop running from 1 to the current row number.
Print the inner loop's counter variable.

Code :

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j);
    }
    System.out.println();
}
```

Diamond

(d) Inverted Triangle :-Logic :

- Use an outer loop to track the current row, running n times.
- Use a first inner loop to print spaces.
- The number of spaces starts at 0 and increases by 1 for each row (i).
- Use a second inner loop to print the number followed by a space ($i + " "$).
- The amount of times the number is printed decreases by 1 each row.
- Move to the next line after both inner loops complete.

Code :

```
for (int i = 3; i <= 7; i++) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
```

```
    for (int j = 0; j < 8 - i; j++) {
        System.out.print(i + " ");
    }
```

```
    System.out.println();
}
```

(e) Sandwich Pattern :-

Logic :

- Split the logic into two parts : an inverted upper triangle (from 3 down to 5) and a regular triangle underneath it (from 4 down to 3).
- The outer loop runs i from 3 to 5, in the case of upper half.
- The outer loop runs backwards, with i from 4 down to 3, in the case of lower half.

Code :

```

for (int i = 3; i <= 5; i++) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * (5 - i) + 1; j++) {
        System.out.print(i + " ");
    }
    System.out.println();
}

```

```

for (int i = 4; i >= 3; i--) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
}

```

```

for (int i = 0; i < 2 * (5 - i) + 1; i++) {
    System.out.print(i + " ");
}
System.out.println();
}

```

(f) Incrementing Diamond Pattern :-

Logic :

- Notice that the numbers are aligned to the left, but they have spaces between them. Split the pattern into an expanding top half and a shrinking bottom half.
- The outer loop runs i from 3 to 1 i.e. upper half.
- The outer loop runs backwards, with i from 6 down to 3 i.e. lower half.

Code :

```

for (int i = 3; i <= 7; i++) {
    for (int j = 0; j < i - 2; j++) {
        System.out.print(i + " ");
    }
    System.out.println();
}

```

```

for (int i = 6; i >= 3; i--) {
    for (int j = 0; j < i - 2; j++) {

```

Date :

Page No.

```
{
    System.out.print(i + ". ");
}
System.out.println();
}
```

(i) Generating Fibonacci series using while loop

Output

At first, we will declare two variables, a and b , and assign them the values 0 and 1 respectively. Then, we will use a while loop to generate the series. In each iteration, we will print the value of a , then update a to the value of b , and update b to the sum of a and b . We will continue this process until we reach the desired number of terms.

For example, to generate the first 10 terms of the Fibonacci series, we will use the following code:

Code

```
{
    int a = 0, b = 1, sum = 0, i = 1;
    while (i <= 10)
    {
        sum = a + b;
        a = b;
        b = sum;
        System.out.print(sum + " ");
        i++;
    }
}
```

```
{
    int a = 0, b = 1, sum = 0, i = 1;
    while (i <= 10)
    {
        sum = a + b;
        a = b;
        b = sum;
        System.out.print(sum + " ");
        i++;
    }
}
```

OUTPUT

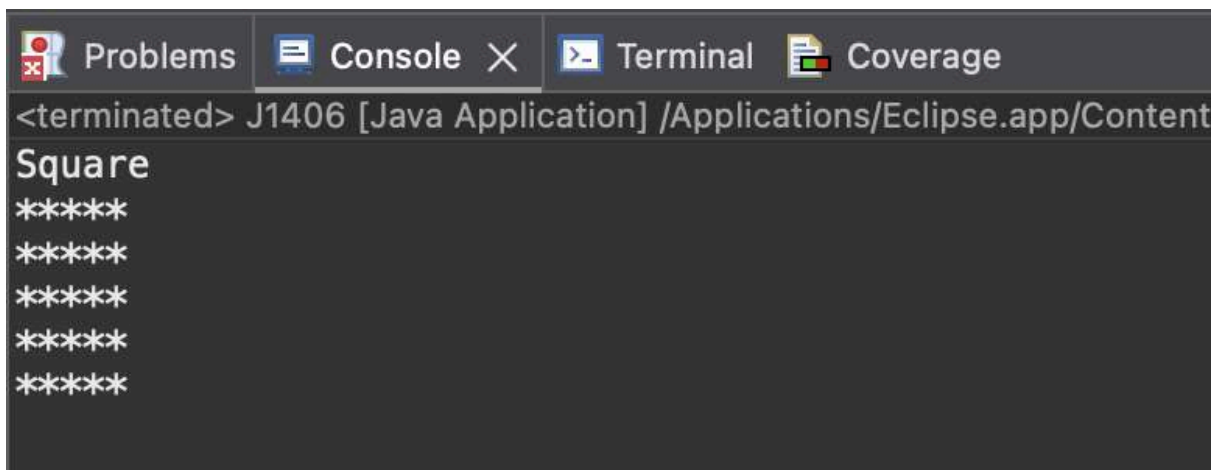
1. STAR PATTERNS:

- SQUARE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the program is displayed as follows:

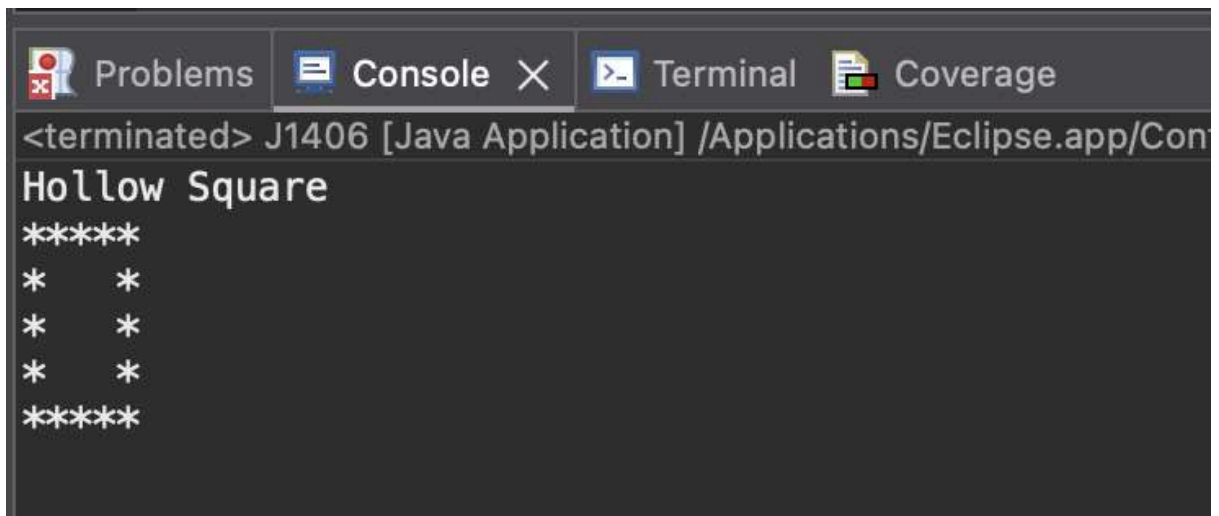
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Content  
Square  
*****  
*****  
*****  
*****  
*****
```

- **HOLLOW SQUARE:**

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Hollow Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {  
                    System.out.print("*");  
                } else {  
                    System.out.print(" ");  
                }  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console tab is active, displaying the output of the Java application J1406. The output consists of the text "Hollow Square" followed by a 5x5 hollow square pattern of asterisks. The pattern is as follows:

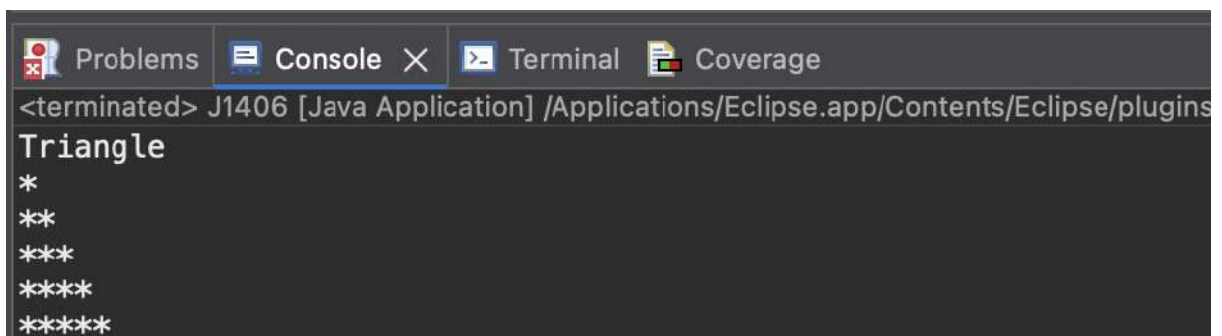
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Con  
Hollow Square  
*****  
*      *  
*      *  
*      *  
*****
```

- TRIANGLE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Triangle");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console tab is active, displaying the output of the Java application. The output starts with the text "<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins", followed by a newline and the word "Triangle". Below this, five lines of asterisks are printed, forming a right-angled triangle: the first line has 1 asterisk, the second has 2, the third has 3, the fourth has 4, and the fifth has 5.

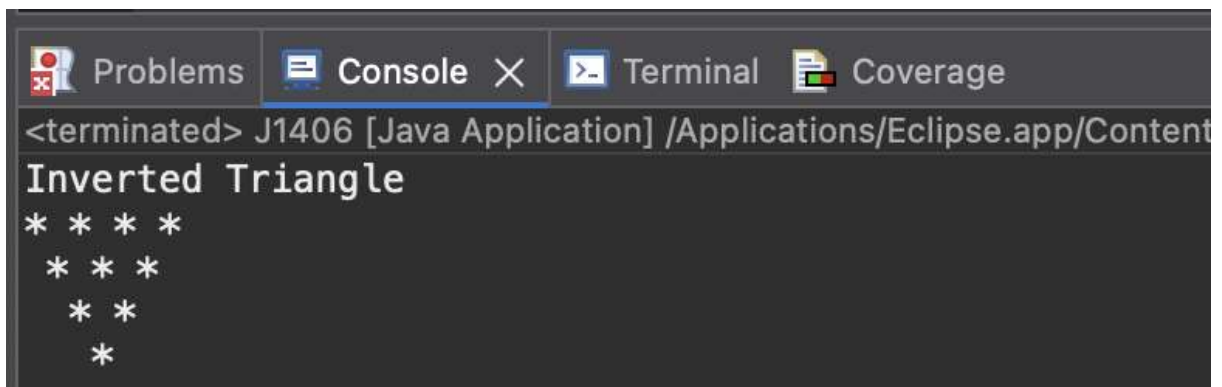
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins  
Triangle  
*  
**  
***  
****  
*****
```

- INVERTED TRIANGLE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Inverted Triangle");  
        int n = 5;  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < (n - i) - 1; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes 'Problems', 'Console', 'Terminal', and 'Coverage'. The console output shows the program's execution: it prints '<terminated> J1406 [Java Application] /Applications/Eclipse.app/Content' followed by 'Inverted Triangle'. Below this, an inverted triangle of asterisks is displayed, consisting of four rows: the first row has 4 asterisks, the second has 3, the third has 2, and the fourth has 1.

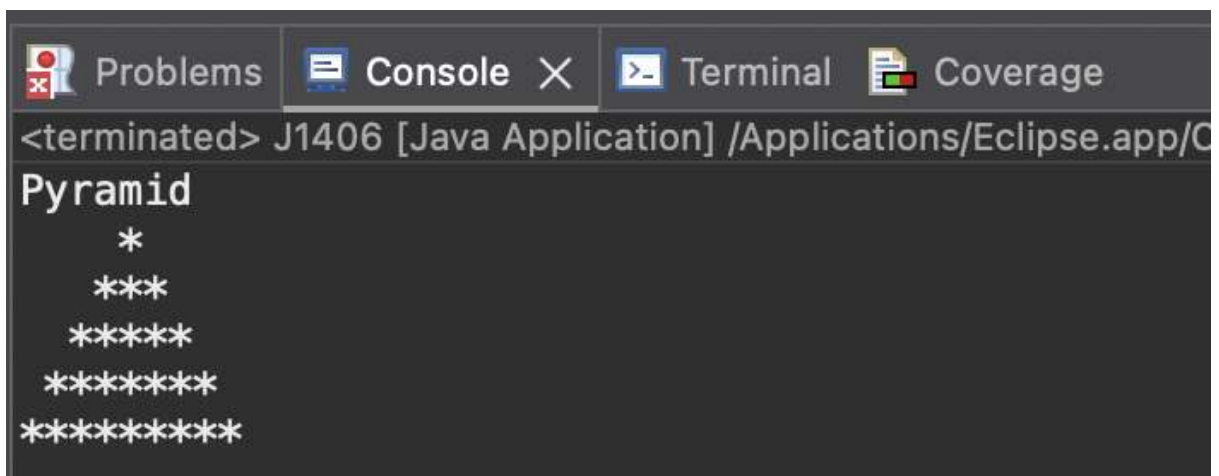
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Content  
Inverted Triangle  
* * * *  
 * * *  
  * *  
   *
```

- PYRAMID:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Pyramid");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n - i - 1; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * i + 1; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the Console tab selected. The console output displays the word "Pyramid" followed by a pyramid shape made of asterisks. The pyramid has 5 rows, with the number of asterisks increasing by 2 in each row (1, 3, 5, 7, 9). There are spaces before the asterisks in each row to center them.

```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/C  
Pyramid  
    *  
   ***  
  *****  
 *****  
*****
```

- DIAMOND:

CODE:

```
public class J1406 {
    public static void main(String[] args) {
        System.out.println("Diamond");
        int n = 5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                System.out.print(" ");
            }
            for (int j = 0; j < 2 * i + 1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j <= i; j++) {
                System.out.print(" ");
            }
            for (int j = 0; j < 2 * (n - i - 1) - 1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

OUTPUT:

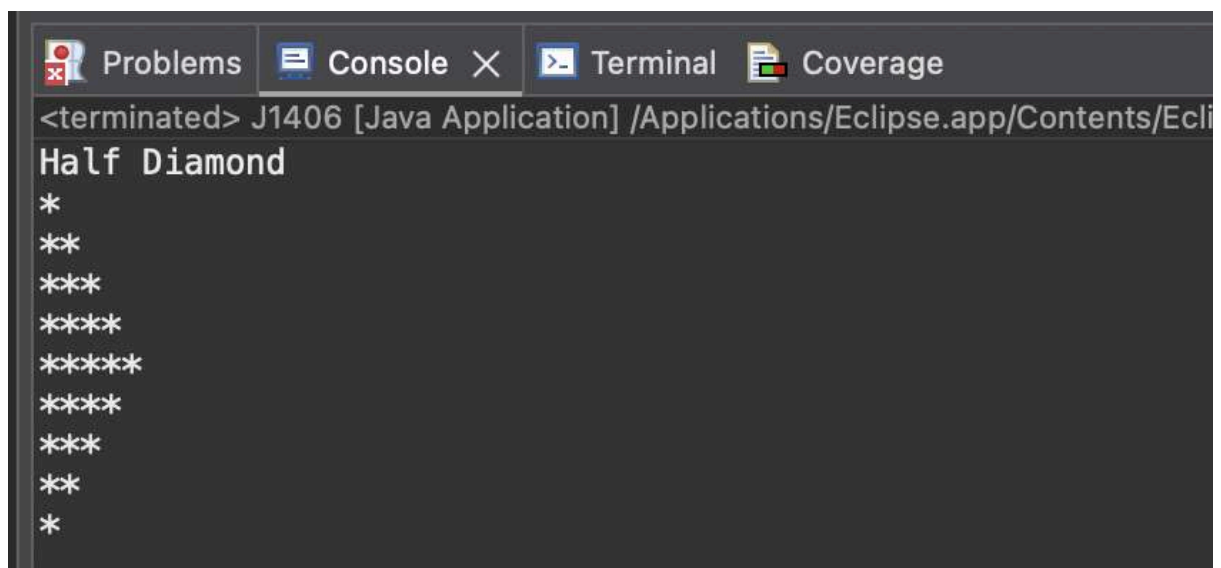
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.e
Diamond
  *
 ***
*****
*****
*****
*****
 *****
  *****
   *****
    *****
     *
```

- HALF DIAMOND:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Half Diamond");  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
        for (int i = n - 1; i >= 1; i--) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console tab is active, displaying the output of the Java application J1406. The output text is "Half Diamond" followed by a diamond shape made of asterisks. The diamond has a height of 10 rows. The first 5 rows form the upper half, and the next 5 rows form the lower half. The number of asterisks in each row corresponds to the row index (1-indexed from the top): 1, 2, 3, 4, 5, 4, 3, 2, 1.

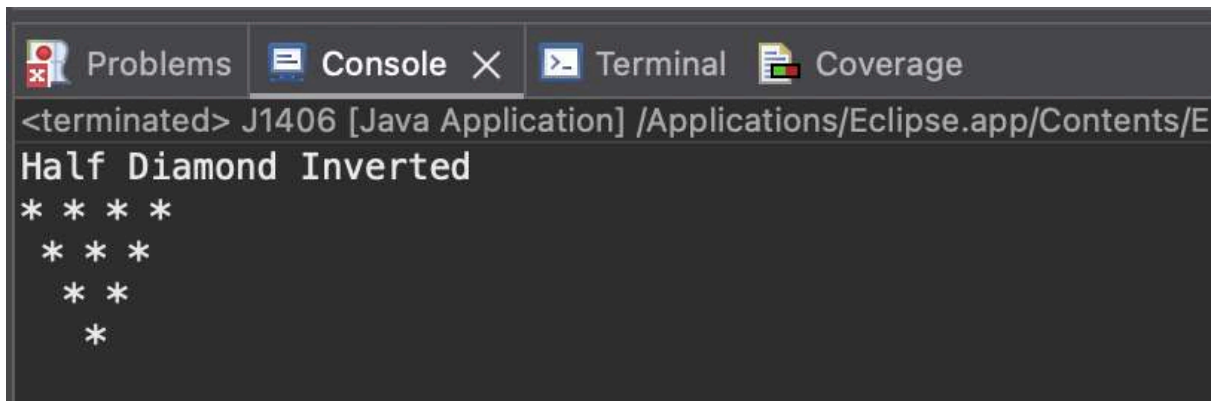
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Ecli  
Half Diamond  
*  
**  
***  
****  
*****  
****  
***  
**  
*
```

- HALF DIAMOND INVERTED:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Half Diamond Inverted");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < (n - i) - 1; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application is displayed, starting with the title 'Half Diamond Inverted' followed by a pattern of asterisks. The pattern consists of four lines: the first line has four asterisks, the second has three, the third has two, and the fourth has one. The asterisks are left-aligned in each line.

```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/E  
Half Diamond Inverted  
* * * *  
 * * *  
  * *  
   *
```

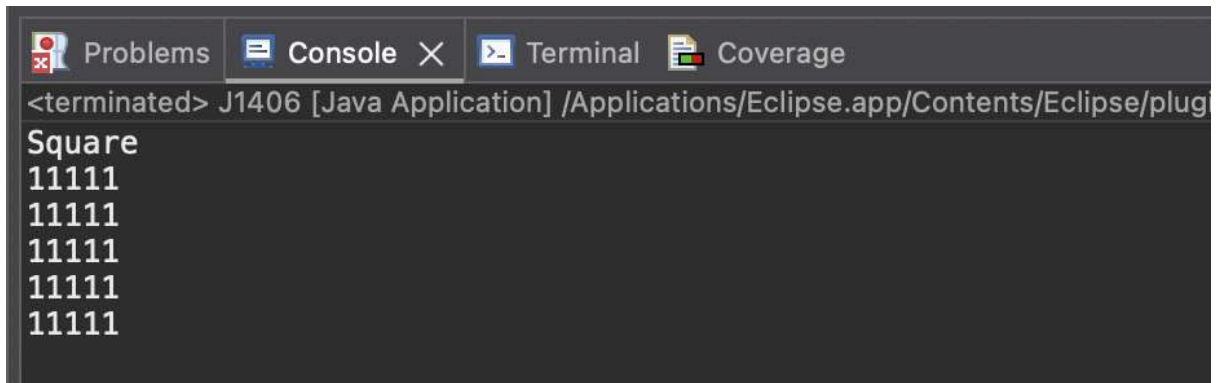
2. NUMBER PATTERNS:

- SQUARE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                System.out.print("1");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console output shows the program's execution: it prints "Square" followed by five lines of "11111", representing a 5x5 square of ones.

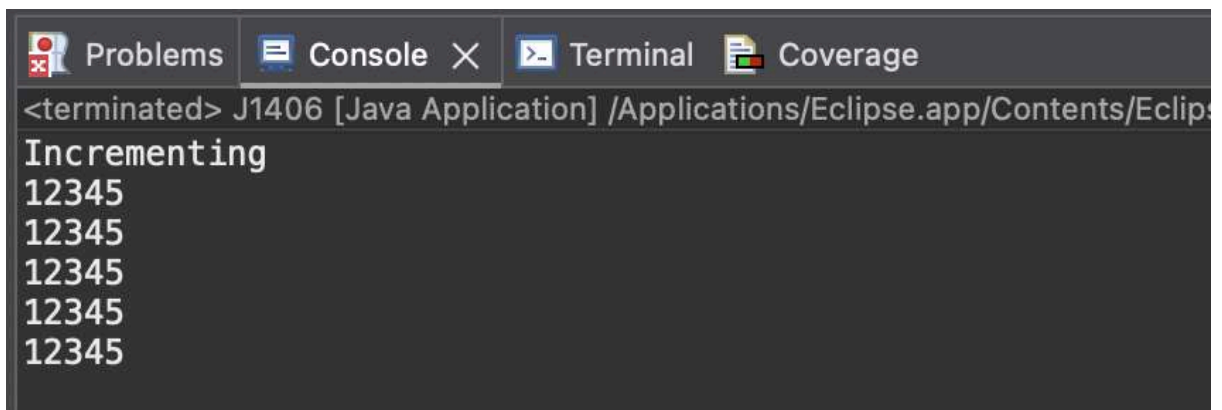
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plug  
Square  
11111  
11111  
11111  
11111  
11111  
11111
```

- INCREMENTING:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Incrementing");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 1; j <= n; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the program is displayed as follows:

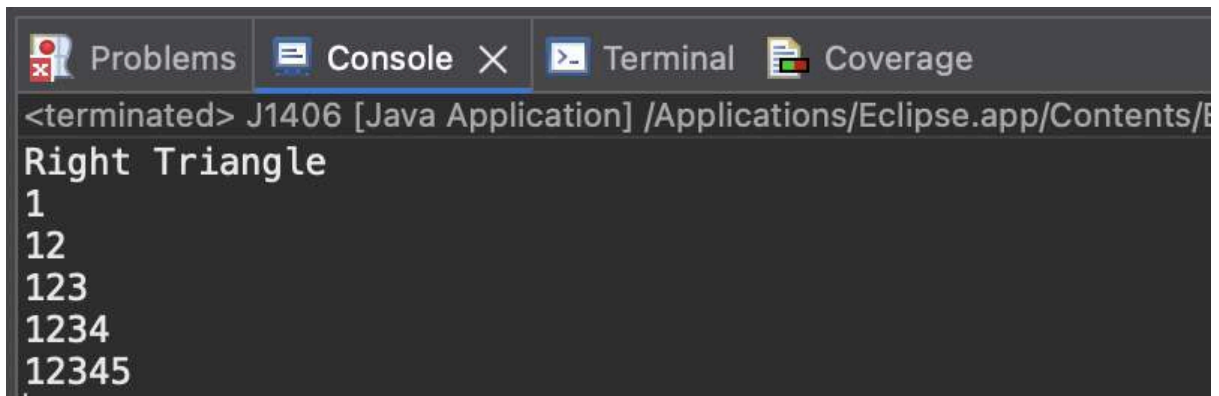
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclips  
Incrementing  
12345  
12345  
12345  
12345  
12345
```

- **RIGHT TRIANGLE:**

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Right Triangle");  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application is displayed, starting with the text 'Right Triangle' followed by five lines of numbers forming a right-angled triangle: '1', '12', '123', '1234', and '12345'. The console title bar indicates the application is 'J1406 [Java Application]' and the path is '/Applications/Eclipse.app/Contents/E'.

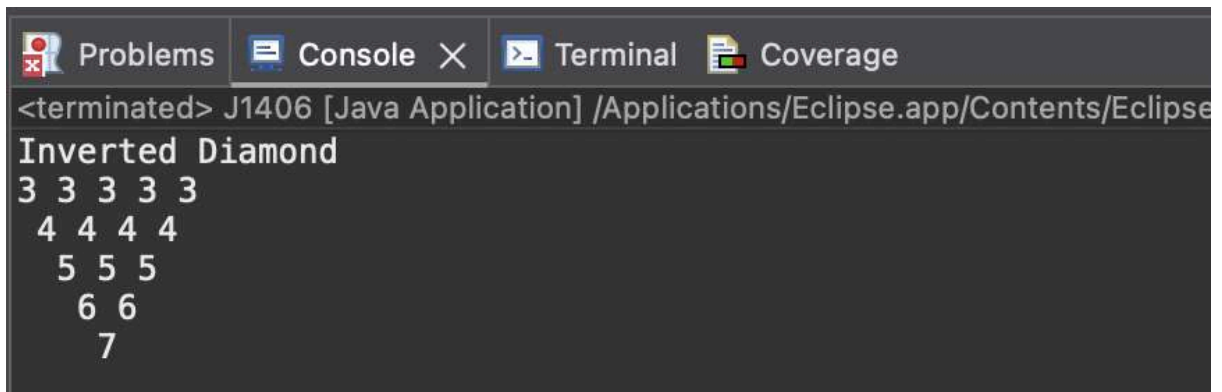
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/E  
Right Triangle  
1  
12  
123  
1234  
12345
```

- INVERTED DIAMOND:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Inverted Diamond");  
        for (int i = 3; i <= 7; i++) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 8 - i; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application is displayed, showing the text 'Inverted Diamond' followed by five lines of numbers forming an inverted diamond shape. The numbers are 3, 4, 5, 6, and 7, with the count of numbers per line decreasing from 5 to 1.

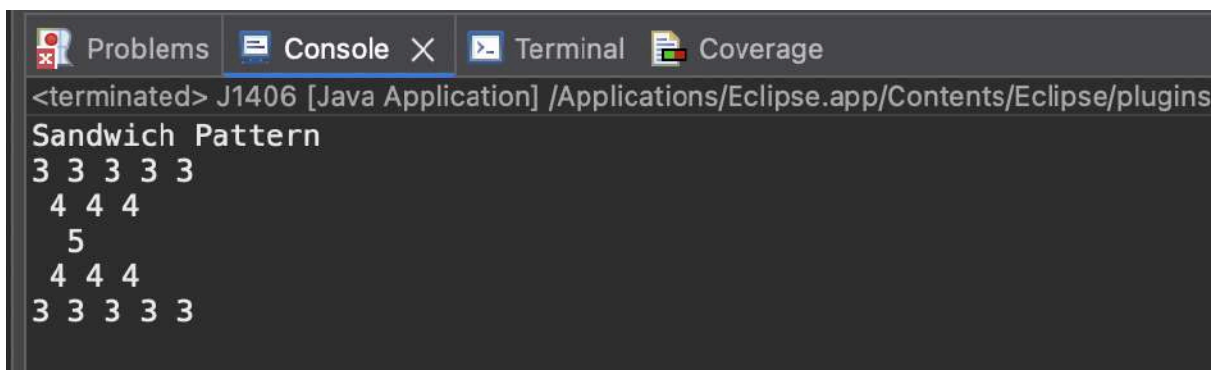
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse  
Inverted Diamond  
3 3 3 3 3  
4 4 4 4  
5 5 5  
6 6  
7
```

- SANDWICH PATTERN:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Sandwich Pattern");  
        for (int i = 3; i <= 5; i++) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * (5 - i) + 1; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
        for (int i = 4; i >= 3; i--) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * (5 - i) + 1; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application is displayed, showing the title 'Sandwich Pattern' followed by a pattern of numbers. The pattern consists of two rows of 5s, each flanked by two rows of 4s, all enclosed within a border of 3s.

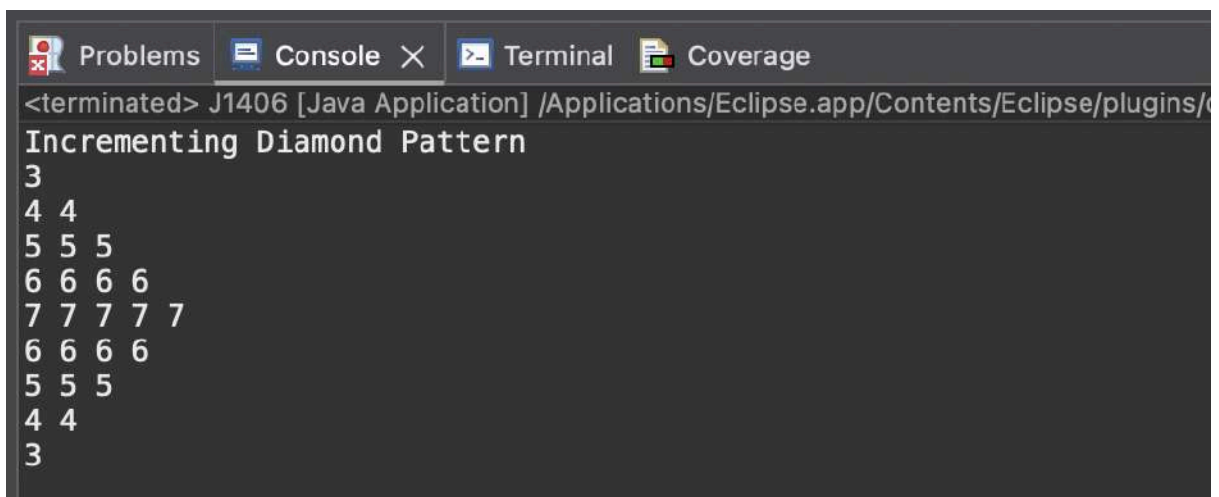
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins  
Sandwich Pattern  
3 3 3 3 3  
 4 4 4  
  5  
 4 4 4  
3 3 3 3 3
```

- INCREMENTING DIAMOND PATTERN:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Incrementing Diamond Pattern");  
        for (int i = 3; i <= 7; i++) {  
            for (int j = 0; j < i - 2; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
        for (int i = 6; i >= 3; i--) {  
            for (int j = 0; j < i - 2; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the Console view active. The output of the Java program is displayed, showing the title "Incrementing Diamond Pattern" followed by a diamond-shaped pattern of the number 3. The pattern consists of 11 lines: a single '3', followed by two '3's, three '3's, four '3's, five '3's, four '3's, three '3's, two '3's, and a final single '3'.

```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/c  
Incrementing Diamond Pattern  
3  
4 4  
5 5 5  
6 6 6 6  
7 7 7 7 7  
6 6 6 6  
5 5 5  
4 4  
3
```